BSc (Hons) Artificial Intelligence and Data Science

Module: CM2604 Machine Learning

Coursework Report

Year	**:**	**02**
Semester	**:**	**01**
Module Coordinator	**:**	**Mr. Sahan Priyanayana**
RGU Student ID	**:**	**2330928**
IIT Student ID	**:**	**20231061**
Student Name	**:**	**Vinuji Dilanya Hewapathirana**

Table of Contents

Chapter 01 - Introduction

This report on machine learning reviews the implementation and assessment of machine learning models created to predict whether a client will subscribe to a term deposit, based on a bank marketing dataset. The models implemented for this requirement include Random Forest Classification and Neural Network, with the final results developed from performance comparison.

Git Repository Link –

https://github.com/Vinuji-Hewapathirana/Machine-Learning---Coursework

Chapter 02 - Corpus Preparation

The bank marketing dataset used for this machine learning coursework consists of client information, including demographic, campaign-related, and historical interaction data.

2.1 Preprocessing

1. **Feature Removal:**

    - The 'duration' column was dropped from the dataset as it seemed less important for the final result.
    - The 'contact' column, which represents the mode of communication, was removed due to its limited predictive value and less importance for the final result.

2. **Managing Missing Values:**

    - Missing values for categorical features (e.g., job, education, default) were replaced with the mode of each category.
    - For numerical features, missing values were replaced using median values to avoid bias.

3. **Encoding Categorical Variables:**

    - One-hot encoding is applied to categorical features, such as job, marital, and poutcome, and transformed into a machine-readable format.

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
```

4. **Normalization:**

    - Scaled all numerical features (e.g., age, campaign, euribor3m) to a range of [0,1] using Min-Max Scaling for faster convergence in the Neural Network model. (StandardScaler has been used)

```
# Create preprocessing transformers
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
```

5. **Addressing Class Imbalance:**

    - Because the target variable y was imbalanced, SMOTE technique was used to balance the classes during training. (Additionally, class weights were also applied in another model that was used as a testing)

2.2 Feature Selection and Reduction

- **Feature Selection:** After completing the initial pipeline, the feature importances are extracted. Based on the feature importance scores from the trained model, threshold (importance > 0.01) has been applied to select important features.
- **Feature Reduction:** After feature selection, the number of features used for training has been reduced by retaining only the important features (emp.var.rate, cons.price.idx, and nr.employed) in the model.
- Principal Component Analysis (PCA) was not used here as the dimensionality of the dataset was manageable.

Chapter 03 - Solution methodology

3.1 Pipeline Overview

1. **Data Splitting:**

    - Divided the dataset into 80% training and 20% testing sets.

```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)
```

2. **Random Forest Model:**

    - Implemented using scikit-learn.
    - The hyperparameter is set to ‘n_estimators=100’, which shows the number of trees.

```
# Step 1: Initial Pipeline with SMOTE and RandomForestClassifier
initial_pipeline = ImbPipeline(steps=[
    ('preprocessor', preprocessor),    # Preprocessing
    ('smote', smote),                  # SMOTE oversampling
    ('classifier', RandomForestClassifier(random_state=42, n_estimators=100))  # Random Forest
])
```

3. **Neural Network Model:**

    - Built using TensorFlow/Keras.
    - Tested using layers with different numbers of neurons (128, 64, 32) and different dropout rates (0.3, 0.2).

4. **Oversampling:**

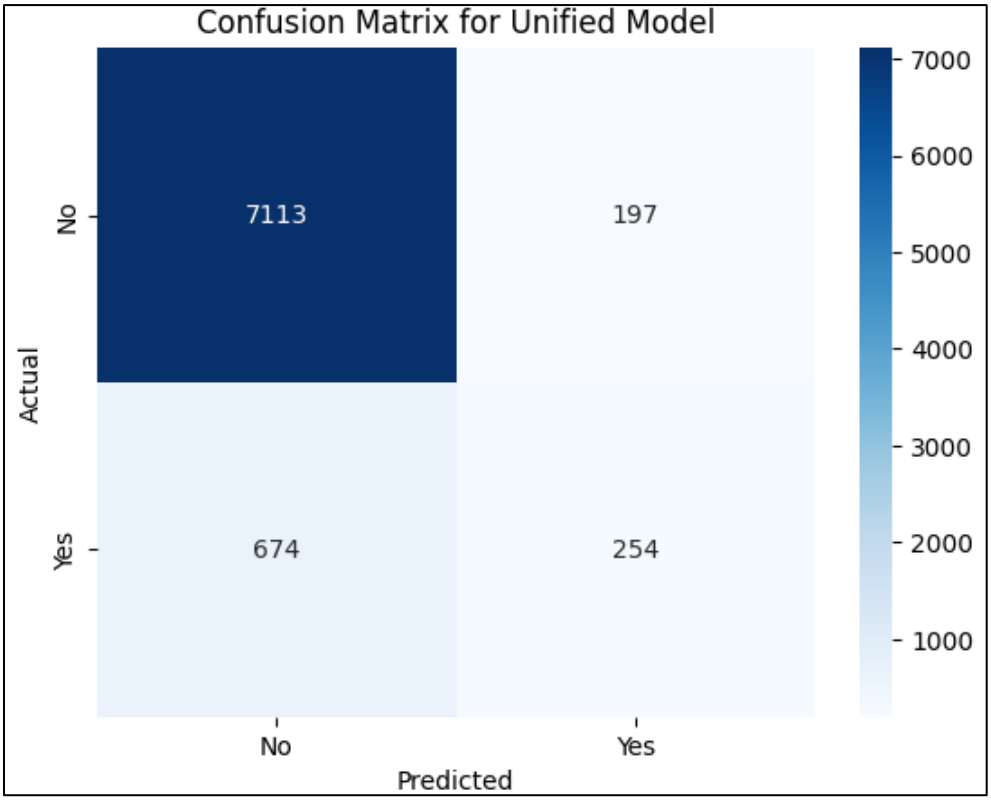
    - Applied SMOTE (Random Forest)
    - Applied Class weights (Neural Network) to address class imbalance

Chapter 04 - Evaluation criteria

The following metrics were used to evaluate the random forest and neural network models:

- **Accuracy:** Overall correctness.
- **Precision and Recall:** Gauge false positives and false negatives in imbalanced datasets.
- **F1-Score:** Combines precision and recall and provide a single metric for model evaluation.
- **ROC-AUC:** Evaluates model performance across all classification thresholds.
- **Macro Average:** Average of Precision, Recall, and F1-score for each class, without considering class imbalance.
- **Weighted Average:** Average of Precision, Recall, and F1-score, weighted by the number of instances in each class.
- **Confusion Matrix**

  True Positives (TP): Correctly predicted positive cases.
  True Negatives (TN): Correctly predicted negative cases.
  False Positives (FP): Incorrectly predicted positive cases (Type I Error).
  False Negatives (FN): Incorrectly predicted negative cases (Type II Error).



Chapter 05 - Model evaluation

5.1 Random Forest Configurations

1. **Default Model with no techniques:**

    - n_estimators = 100

```
Classification Report
              precision    recall  f1-score   support

           0       0.91      0.97      0.94      7310
           1       0.56      0.28      0.37       928

    accuracy                           0.89      8238
   macro avg       0.73      0.62      0.66      8238
weighted avg       0.87      0.89      0.88      8238

ROC-AUC Score: 0.7729383608896645
```

2. **Model with SMOTE:**

    - n_estimators = 100

```
Classification Report with SMOTE
              precision    recall  f1-score   support

          no       0.92      0.95      0.94      7310
         yes       0.49      0.34      0.40       928

    accuracy                           0.89      8238
   macro avg       0.70      0.65      0.67      8238
weighted avg       0.87      0.89      0.88      8238

ROC-AUC Score with SMOTE: 0.780920680220765
```

3. **Model with class weights:**

    - n_estimators = 100 , class_weights = balanced
    - Returns of Class 1 remain low.

```
Classification Report with Class Weights
              precision    recall  f1-score   support

           0       0.91      0.97      0.94      7310
           1       0.56      0.27      0.36       928

    accuracy                           0.89      8238
   macro avg       0.74      0.62      0.65      8238
weighted avg       0.87      0.89      0.88      8238

ROC-AUC Score with Class Weights: 0.7710959980423605
```

4. **Model with feature reduction:**

    - n_estimators = 100
    - Performance gains are marginal compared to the class weights strategy.

```
Classification Report
              precision    recall  f1-score   support

           0       0.91      0.97      0.94      7310
           1       0.56      0.28      0.37       928

    accuracy                           0.89      8238
   macro avg       0.73      0.62      0.66      8238
weighted avg       0.87      0.89      0.88      8238

ROC-AUC Score: 0.7729383608896645
```

5. **Model with hyper parameter tuning:**

    - n_estimators = 200, max_depth = 10, min_samples_leaf = 1, min_Samples_split = 5

```
Best ROC-AUC Score on Training Data: 0.790981413174164
Classification Report
              precision    recall  f1-score   support

           0       0.91      0.99      0.95      7310
           1       0.71      0.21      0.32       928

    accuracy                           0.90      8238
   macro avg       0.81      0.60      0.64      8238
weighted avg       0.89      0.90      0.88      8238

ROC-AUC Score: 0.8072476148639087
```

6. **Final Model with both SMOTE and feature reduction:**

    - n_estimators = 100
    - This configuration provided the best balance between performance and computational efficiency.

```
Classification Report for Unified Model
              precision    recall  f1-score   support

           0       0.91      0.97      0.94      7310
           1       0.56      0.27      0.37       928

    accuracy                           0.89      8238
   macro avg       0.74      0.62      0.66      8238
weighted avg       0.87      0.89      0.88      8238

ROC-AUC Score for Unified Model: 0.7767726366809755
```

5.2 Neural Network Configurations

1. **Model with SMOTE:**

    - 3 hidden layers with 64,32,32 neurons each, ReLU activation, Adam optimizer, learning rate = 0.001.

```
Classification Report (Neural Network)
              precision    recall  f1-score   support

           0       0.94      0.90      0.92      7310
           1       0.40      0.51      0.45       928

    accuracy                           0.86      8238
   macro avg       0.67      0.71      0.69      8238
weighted avg       0.88      0.86      0.87      8238

ROC-AUC Score (Neural Network): 0.7695040892259067
```

2. **Final Model with class weights:**

    - 3 hidden layers, 128, 64, 32 neurons each, dropout (0.3), (0.2), Adam optimizer, learning rate = 0.0001, early Stopping: Patience of 5 epochs.
    - The final configuration has improved generalization and managed class imbalance effectively.

```
Classification Report (Neural Network with Class Weights
              precision    recall  f1-score   support

           0       0.95      0.88      0.92      7310
           1       0.41      0.64      0.50       928

    accuracy                           0.86      8238
   macro avg       0.68      0.76      0.71      8238
weighted avg       0.89      0.86      0.87      8238

ROC-AUC Score (Neural Network): 0.8049669206094627
```

Chapter 06 - Experimental results

The two models were analyzed based on performance metrics such as precision, recall, F1-score, accuracy, ROC-AUC score, and confusion matrix analysis.

6.1 Random Forest Model

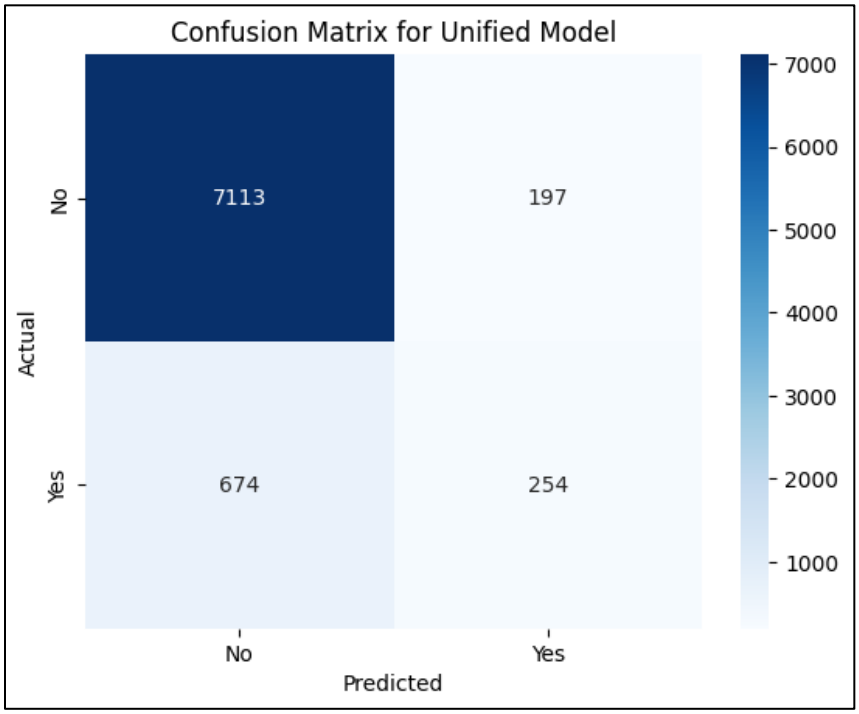
Classification Report:

```
Classification Report for Unified Model
              precision    recall  f1-score   support

           0       0.91      0.97      0.94      7310
           1       0.56      0.27      0.37       928

    accuracy                           0.89      8238
   macro avg       0.74      0.62      0.66      8238
weighted avg       0.87      0.89      0.88      8238

ROC-AUC Score for Unified Model: 0.7767726366809755
```



Feature Importance:

- The model discovered key features such as euribor3m, campaign, age, and nr.employed as the important features for the target variable ‘y’, where euribor3m is the highest contributing factor.

Interpretation:

- The Random Forest model with SMOTE and feature reduction performed well in predicting the negative class (No), with a high recall of 0.97 and precision of 0.91. Conversely, it struggled with the positive class (Yes), having a recall of only 0.27 and a precision of 0.56. The ROC-AUC score of 0.7767 shows that the model has reasonable discriminatory ability.

6.2 Neural Network Model

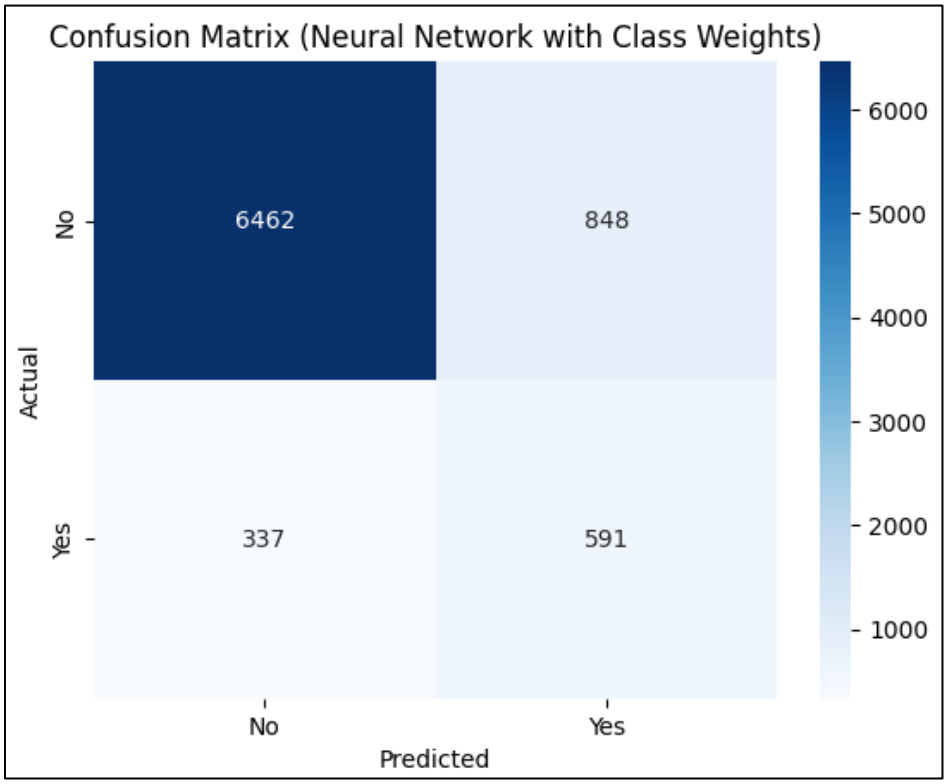
Classification Report:

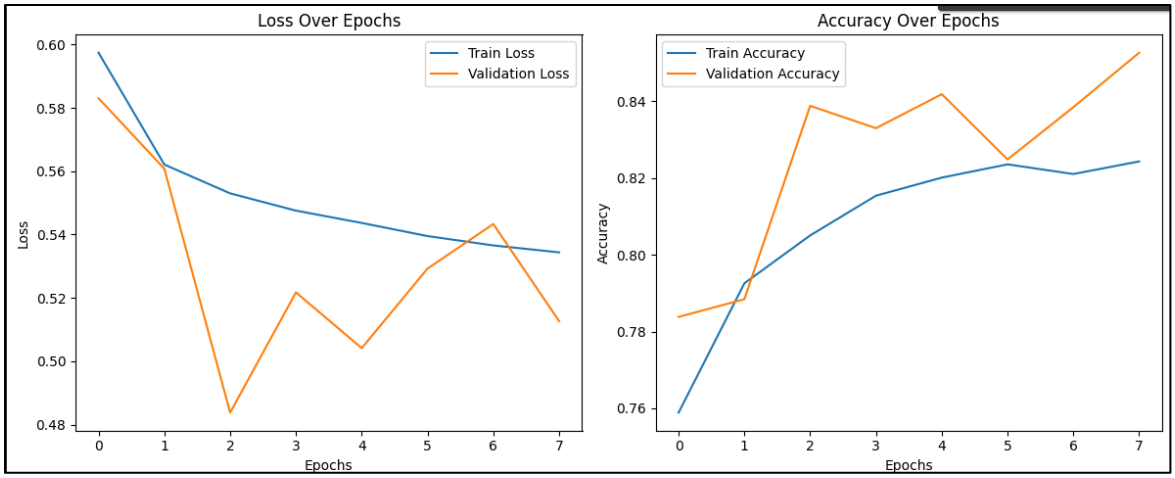
```
Classification Report (Neural Network with Class Weights)
              precision    recall  f1-score   support

           0       0.95      0.88      0.92      7310
           1       0.41      0.64      0.50       928

    accuracy                           0.86      8238
   macro avg       0.68      0.76      0.71      8238
weighted avg       0.89      0.86      0.87      8238

ROC-AUC Score (Neural Network): 0.8049669206094627
```





The above provided charts visualize the training and validation performance of the neural network. According to the Loss Over Epochs (Left Chart):

- The gap between training and validation loss shows that the model generalizes reasonably well.
- Both training and validation losses decrease initially, showcasing that the model is learning and improving its ability to predict correctly.

According to the Accuracy Over Epochs (Right Chart):

- Training accuracy steadily improves over time, showing the model is fitting the training data well.
- The validation accuracy surpasses training accuracy early, due to the effects of batch normalization.

Model Architecture and Training:

- The Neural Network utilized **128**, **64**, and **32**-node layers with **Batch Normalization** and **Dropout** to improve training and generalization. The **class weights** were used to handle the class imbalance, which made the model to pay more attention to the minority class (Yes).

Interpretation:

- The Neural Network model achieved a higher recall for the positive class (Yes) at 0.64 compared to the Random Forest's recall of 0.27. This shows the Neural Network is better at identifying positive cases. However, the precision for the positive class (Yes) is lower than that of the Random Forest model, which indicates that the model made more false positives. The ROC-AUC score of 0.8050 of the Neural Network model is slightly better than the Random Forest's score of 0.7767.

6.3 Comparison and Best Model Selection:

Metric	Random Forest	Neural Network
Precision (Class 0)	0.91	0.95
Precision (Class 1)	0.56	0.41
Recall (Class 0)	0.97	0.88
Recall (Class 1)	0.27	0.64
F1-Score (Class 0)	0.94	0.92
F1-Score (Class 1)	0.37	0.50
Accuracy	0.89	0.86
ROC-AUC Score	0.7768	0.8050

Key Observations:

- The **Random Forest model** is good in precision and recall for the negative class (No) but struggles with the positive class (Yes). This results in a high F1-score for the negative class and lower F1-score for the positive class.
- The **Neural Network model** shows a higher recall for the positive class (Yes), showing it is better at identifying the minority class
- The **ROC-AUC score** of the Neural Network is slightly better than the Random Forest model.

Conclusion:

The **Neural Network model** is the best-performing model for this task. Despite its lower precision for the positive class, it provides better recall and overall performance in terms of distinguishing between the two classes.

Chapter 07 - Limitations

- **Class Imbalance:** Regardless of oversampling, the minority class (subscribed clients) presented challenges for prediction.
- **Computational Time:** Time for Neural Network training was a little higher, especially with larger architecture.
- **Feature Engineering:** Limited usage of advanced feature engineering techniques, which may have enhanced model performance.

Chapter 08 - Further enhancements

- **Hyperparameter Tuning:** Can use automated tools like Grid Search or Bayesian Optimization for better tuning.
- **Cross-Validation**: Can use k-fold cross-validation to ensure stability in validation performance and reduce sensitivity to the data split. (Neural Network)
- **Additional Features:** Can include external economic indicators (e.g., interest rates) to improve prediction accuracy.
- **Alternative Models:** Can explore other machine learning models like Gradient Boosting or hybrid models combining Neural Networks and Random Forests.

Chapter 09 – Appendix

9.1 Source Code for Random Forest Model

```
import pandas as pd
# Load data
data = pd.read_csv('bank-additional-full.csv', sep=';')
data.head()

# Drop the 'duration', ' contact'  columns
data = data.drop(columns=['duration','contact'])

# Display the first few rows to verify
data.head()
#Prepare features and target
X = data.drop(columns=['y'])
y = data['y']

print(f"Input features (X): {X.shape}")
print(f"Target variable (y): {y.shape}")

#encoding the y attribute
y_encoded = y.map({'yes':1, 'no':0})
print(y_encoded.head())
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline  # Use imbalanced-learn's Pipeline
import seaborn as sns
import matplotlib.pyplot as plt

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
   X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

# Identify numeric and categorical features
numeric_features = X.select_dtypes(include=['number']).columns
```

```
categorical_features = X.select_dtypes(include=['object']).columns

# Create preprocessing transformers
numeric_transformer = Pipeline(steps=[
   ('imputer', SimpleImputer(strategy='mean')),
   ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
   ('imputer', SimpleImputer(strategy='most_frequent')),
   ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
   transformers=[
      ('num', numeric_transformer, numeric_features),
      ('cat', categorical_transformer, categorical_features)
   ]
)

# Create the SMOTE object
smote = SMOTE(random_state=42)

# Step 1: Initial Pipeline with SMOTE and RandomForestClassifier
initial_pipeline = ImbPipeline(steps=[
   ('preprocessor', preprocessor),   # Preprocessing
   ('smote', smote),              # SMOTE oversampling
   ('classifier', RandomForestClassifier(random_state=42, n_estimators=100))  # Random Forest
])

# Train the pipeline
initial_pipeline.fit(X_train, y_train)

# Step 2: Feature Importance Extraction
model = initial_pipeline.named_steps['classifier']
categorical_feature_names =
initial_pipeline.named_steps['preprocessor'].named_transformers_['cat']['onehot'].get_feature_names_
out(categorical_features)
feature_names = np.concatenate([numeric_features, categorical_feature_names])

# Get feature importances
feature_importances = model.feature_importances_
```

```
# Create a DataFrame for feature importance
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': feature_importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Select features with importance above a threshold
important_features = importance_df[importance_df['Importance'] > 0.01]['Feature'].values

# Transform the dataset using the preprocessor
X_train_transformed = initial_pipeline.named_steps['preprocessor'].transform(X_train)
X_test_transformed = initial_pipeline.named_steps['preprocessor'].transform(X_test)

# Convert the transformed datasets to DataFrames
X_train_transformed = pd.DataFrame(X_train_transformed, columns=feature_names)
X_test_transformed = pd.DataFrame(X_test_transformed, columns=feature_names)

# Filter the reduced dataset
X_train_reduced = X_train_transformed[important_features]
X_test_reduced = X_test_transformed[important_features]

# Step 3: Final Pipeline with Reduced Features
final_pipeline = ImbPipeline(steps=[
   ('classifier', RandomForestClassifier(random_state=42, n_estimators=100))
])

# Train the final model
final_pipeline.fit(X_train_reduced, y_train)

# Evaluate the final model
y_pred = final_pipeline.predict(X_test_reduced)
y_prob = final_pipeline.predict_proba(X_test_reduced)[:, 1]

print("Classification Report for Unified Model")
print(classification_report(y_test, y_pred))
print("ROC-AUC Score for Unified Model:", roc_auc_score(y_test, y_prob))

# Visualize the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['No', 'Yes'], yticklabels=['No',
'Yes'])
plt.title("Confusion Matrix for Unified Model")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
# Display top 10 features after reduction
print("\nTop 10 Important Features After Reduction:")
print(importance_df.head(10))

# Visualize feature importance
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df.head(10))
plt.title('Top 10 Feature Importances (Unified Model)')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

9.2 Source Code for Neural Network Model

```
import pandas as pd
# Load data
data = pd.read_csv('bank-additional-full.csv', sep=';')
data.head()
# Drop the 'duration', 'contact' columns
data = data.drop(columns=['duration', 'contact'])

# Display the first few rows to verify
data.head()
#Prepare features and target
#encoding the y attribute
X = data.drop(columns=['y'])
y = data['y'].map({'yes': 1, 'no': 0}).astype(int)

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
from tensorflow.keras.callbacks import EarlyStopping

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Identify numeric and categorical features
numeric_features = X.select_dtypes(include=['number']).columns
categorical_features = X.select_dtypes(include=['object']).columns

# Create preprocessing transformers
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)

# Transform data using preprocessor
X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

# Calculate class weights
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train),
    y=y_train
)

# Correct class_weights_dict construction
class_weights_dict = {cls: weight for cls, weight in zip(np.unique(y_train), class_weights)}
print("Class Weights:", class_weights_dict)
```

```
# Ensure input is dense
import scipy.sparse
if isinstance(X_train_transformed, scipy.sparse.spmatrix):
   X_train_transformed = X_train_transformed.toarray()
   X_test_transformed = X_test_transformed.toarray()

# Ensure y_train is a dense NumPy array
y_train = np.asarray(y_train)
y_test = np.asarray(y_test)

from tensorflow.keras.layers import BatchNormalization

# Define the Neural Network model
input_dim = X_train_transformed.shape[1]
model = Sequential([
   tf.keras.Input(shape=(input_dim,)),  # Specify input shape
   Dense(128, activation='relu'),
   BatchNormalization(),
   Dropout(0.3),
   Dense(64, activation='relu'),
   BatchNormalization(),
   Dropout(0.2),
   Dense(32, activation='relu'),
   Dense(1, activation='sigmoid')  # Binary classification
])

# Compile the model
model.compile(optimizer='adam',
          loss='binary_crossentropy',
          metrics=['accuracy', tf.keras.metrics.AUC(name='auc')])

# Train the model with EarlyStopping
early_stopping = EarlyStopping(
   monitor='val_loss', patience=5, restore_best_weights=True
)

# Train the model
history = model.fit(
   X_train_transformed, y_train,
   validation_data=(X_test_transformed, y_test),
   epochs=50, batch_size=32,
   callbacks=[early_stopping],
   class_weight=class_weights_dict
)
```

```
# Evaluate the model
y_pred_prob = model.predict(X_test_transformed).ravel()
y_pred = (y_pred_prob > 0.5).astype(int)

print("Classification Report (Neural Network with Class Weights)")
print(classification_report(y_test, y_pred))
print("ROC-AUC Score (Neural Network):", roc_auc_score(y_test, y_pred_prob))

# Confusion Matrix
conf_matrix_nn = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix_nn, annot=True, fmt='d', cmap='Blues', xticklabels=['No', 'Yes'],
yticklabels=['No', 'Yes'])
plt.title("Confusion Matrix (Neural Network with Class Weights)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Plot training history
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss Over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()
```

Chapter 10 - References

3Blue1Brown But what is a neural network? | Deep learning chapter 1. Youtube. Available at: https://www.youtube.com/watch?v=aircAruvnKk&pp=ygUkbmV1cmFsIG5ldHdvcmtzIGZvciBtYWNoaW5lIGxlYXJuaW5n (Accessed: December 6, 2024).

codebasics Machine learning tutorial python - 11 random forest. Youtube. Available at:https://www.youtube.com/watch?v=ok2s1vV9XW0&pp=ygUecmFuZG9tIGZvcmVzdCBtYWNoaW5lIGxlYXJuaW5n (Accessed: December 1, 2024).

Datacamp.com. Available at: https://www.datacamp.com/tutorial/tensorflow-tutorial?(Accessed: December 9, 2024).

Naik, K. Tutorial 43-random forest classifier and regressor. Youtube. Available at: https://www.youtube.com/watch?v=nxFG5xdpDto&pp=ygUecmFuZG9tIGZvcmVzdCBtYWNoaW5lIGxlYXJuaW5n (Accessed: December 1, 2024).

Scikit-learn Scikit-learn.org. Available at: https://scikit-learn.org/stable/(Accessed: December 6, 2024).

TutorialsPoint (no date) How neural network works? | neural networks in machine learning. Youtube. Available at: https://www.youtube.com/watch?v=yKZ3uthT2zc&pp=ygUkbmV1cmFsIG5ldHdvcmtzIGZvciBtYWNoaW5lIGxlYXJuaW5n (Accessed: December 9, 2024).